

Learning Decomposed Visibility for Efficient Active Exploration of Cluttered Scenes

Suyun Lee* Minsoo Choi* Jihwan Gong Unghui Nam Minji Bae Byonghyo Shim

Seoul National University
{sylee, mschoi, uhnam, mjb, bshim}@islabs.snu.ac.kr

*Equal Contribution

Abstract: Active exploration in cluttered environments is essential for reconstructing occluded 3D scenes, requiring both observation and physical interaction. Existing methods rely on belief maps that indicate where the scene is unresolved but not how each region can be resolved, estimating the gains of viewing and pushing only through costly lookahead or hand-tuned heuristics. We propose DiVE (Decomposed Visibility Estimator), a learned model that predicts two decomposed visibility maps: a view-resolvable map and a push-resolvable map, making explicit what each action mode can reveal. By converting these modality-specific predictions into a common visibility-gain scale, DiVE supports a hierarchical decision structure: the robot first chooses between observation and interaction, then selects the specific action through a mode-specific policy. In simulation, DiVE improves occupancy IoU by 11.3% on extreme-occlusion shelf scenes and runs $13.2\times$ faster per step ($19.8\times$ faster in planning) than a state-of-the-art baseline. Deployed zero-shot on a real robot, it recovers 79.2% of initially occluded objects.

Keywords: Active exploration, Interactive perception, View-and-push planning, Cluttered scene reconstruction

1 Introduction

Robots are increasingly expected to operate in everyday environments such as warehouses, kitchens, and storage spaces [1, 2, 3, 4, 5, 6]. Reliable manipulation in such environments requires more than recognizing visible objects: a robot must recover the geometric and semantic structure of the scene, including object locations and object categories [7, 8]. However, everyday scenes are often cluttered, with objects densely arranged and mutually occluding one another, so a single observation provides only a partial view of the scene. This partial observability makes active exploration necessary: the robot must acquire information beyond its initial view by moving the camera to informative viewpoints or physically interacting with objects to expose regions that no viewpoint can reach.

Recent methods therefore jointly plan viewpoints and physical interactions over a shared representation of what the scene still hides, typically in the form of a belief or uncertainty map [9, 10]. While such a representation indicates where the scene remains unresolved, it does not specify how each hidden region can be resolved—by moving the camera or by physically changing the scene. As a result, the expected gain of viewing versus pushing is not directly available from the representation itself. Existing methods therefore recover this quantity indirectly, either by precomputing which regions each candidate action would reveal and scoring each action against the current uncertainty, or by relying on hand-tuned switching thresholds. In both cases, action selection is imposed on top of the representation rather than derived from it. This makes the planner either computationally costly or reliant on hand-tuned heuristics, limiting its ability to select informative viewing and pushing actions as scene geometry and occlusion patterns change.

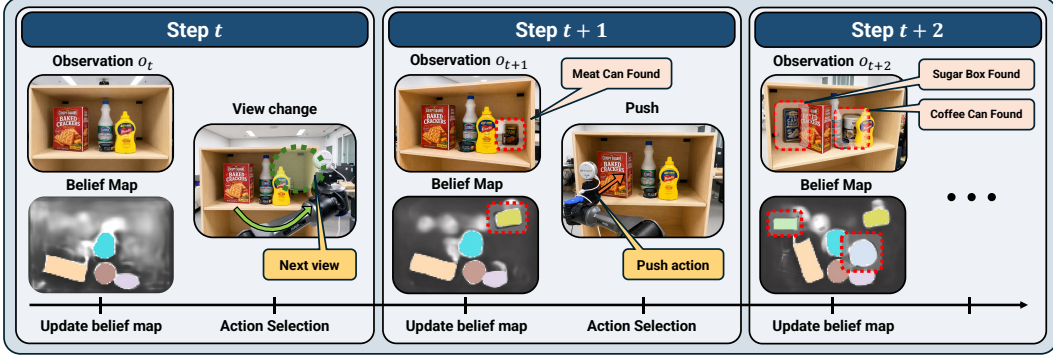


Figure 1: **Example scenario in a real shelf environment.** At each step, the belief map estimator updates the global scene belief from the current observation, visualized here as a 2D top-down projection. Given the visibility maps, the planner decides whether to change the viewpoint or execute a push. In this example, the planner selects a view change at step t and executes a push at step $t + 1$.

We instead center the representation on visibility, the quantity that actions directly change. Rather than asking only where uncertainty remains, we model how hidden regions can become observable. Our key insight is that visibility provides an action-resolvable signal: some regions are occluded only from the current view and can be revealed by moving the camera, whereas others remain hidden from all accessible viewpoints until the scene geometry is changed through interaction.

Building on this insight, we propose the **Decomposed Visibility Estimator (DiVE)**, a learned model that predicts how visibility can be improved under different action modalities. Given the current partial observation, DiVE estimates two complementary maps: a view-resolvable visibility map ϕ_{view} , which captures regions likely to become visible through viewpoint changes, and a push-resolvable visibility map ϕ_{push} , which captures regions likely to become visible after pushing. This decomposition makes the action effect explicit: instead of inferring action gain from uncertainty through external scoring rules, DiVE directly represents what each mode can reveal in the current scene.

We use these decomposed visibility maps within a hierarchical view-and-push planner. Hierarchical planning itself has been explored in prior work; here, the decomposition supplies the planner with a representation-level signal for comparing the two action modes. At each step, the planner converts the modality-specific maps to a common expected-visibility-change scale, selects between viewing and pushing, and then invokes a mode-specific policy to rank the corresponding candidates. This lightweight use of the representation avoids candidate-wise post-action belief prediction and does not require a hand-tuned constant to balance the two modes.

We evaluate DiVE on cluttered shelf scenes in both simulation and the real world, where recovering the geometric and semantic structure of the scene requires both viewpoint changes and pushing.

In summary, our contributions are as follows:

- Our main methodological contribution is the Decomposed Visibility Estimator (DiVE), a learned module that predicts view-resolvable and push-resolvable visibility maps from partial observations. By preserving which action modality can resolve each region, the representation exposes action-relevant structure that is absent from a unified belief or uncertainty map.
- We show how the decomposed maps support efficient planning: modality-specific predictions are converted to a common expected-visibility-change scale for mode selection and are used by lightweight policies for within-mode candidate ranking. Unlike prior hierarchical view-and-push planners [10, 11, 12, 13], the view-versus-push signal follows from the proposed representation rather than candidate-wise post-action belief prediction or a hand-tuned balancing constant.
- We validate our framework in both simulation and real-world cluttered shelf scenes. In simulation, DiVE improves occupancy IoU by 11.3% under extreme occlusion and runs

13.2 $\times$ faster per step than a state-of-the-art baseline. On a real robot, DiVE transfers zero-shot and recovers 79.2% of initially occluded objects.

2 Related Work

2.1 Next-Best-View Planning

Next-best-view (NBV) planning selects viewpoints by maximizing information gain over a volumetric scene representation [14, 15, 16, 17, 18]. Later work reduces ray-casting cost [19] or replaces hand-crafted criteria with learned viewpoint prediction [20, 21, 22]. Even when extended to manipulator-mounted cameras in confined clutter [23], these methods treat the scene as static and assume hidden geometry can be recovered from viewpoints alone [24, 25, 26], either by direct observation or by shape inference. This assumption fails in dense clutter, where some regions remain occluded from every accessible viewpoint unless the scene is physically altered.

2.2 Interactive Perception

Interactive perception [27, 28] extends active perception [29, 30, 31] by allowing the robot to physically alter the scene rather than only sampling it from different viewpoints. Physical interaction has improved perception in tasks such as instance segmentation in dense clutter [32, 33, 34] and retrieving target objects occluded by surrounding items [35, 36, 37, 38]. Across these settings, manipulation is the primary mechanism for revealing hidden information, while the viewpoint is treated as fixed. This fixed-camera assumption motivates planning viewpoint changes and physical interactions within a single decision process.

2.3 Combined Viewpoint and Manipulation Planning

Recent work jointly plans viewpoints and pushes, recognizing that neither modality alone suffices in clutter. Early efforts trained RL policies that switch between the two for object search [11, 12]. On cluttered shelves, Dengler et al. [10] trigger a push only after viewpoint information gain saturates, using hand-tuned thresholds; a follow-up work replaces this heuristic with uncertainty-informed selection [13]. Marques et al. [9] instead formulate the problem as a POMDP in metric-semantic map space and solve it with receding-horizon planning using calibrated neural belief updates. Despite these advances, existing methods do not provide a representation-level gain signal that makes viewing and pushing directly comparable. Consequently, action selection still relies on expensive lookahead [9] or heuristic mode switching [10]. DiVE retains the established hierarchical view-and-push structure, but replaces a unified unresolved-scene representation with modality-conditioned visibility maps. The resulting representation provides the common-scale gain signal used by the planner without requiring full post-action belief prediction or a hand-tuned cross-mode threshold.

3 Problem Definition

We address active scene reconstruction in cluttered shelf environments, where a robot must recover geometric and semantic scene structure from partial observations. Due to the shelf’s spatial constraints, we consider two complementary action types: viewpoint selection for sensing around occlusions and pushing for altering occlusion geometry. The action space is $\mathcal{A} = \mathcal{C} \cup \mathcal{P}$, where $\mathcal{C}$ contains candidate viewpoints and $\mathcal{P}$ contains candidate pushes parameterized by contact point and direction. The scene is represented as a voxel grid $\mathcal{V}$ with labels $z \in \{0, 1, \dots, N_{\text{classes}}\}$ [39, 40], where 0 denotes free space and the remaining labels denote object categories. The robot maintains a belief $\phi_{\text{scene},t}$ over the current scene state s_t , and the objective is to reconstruct the scene accurately while selecting informative actions efficiently at each step.

4 Methodology

4.1 Overview

Our method separates two questions that are often coupled in active scene reconstruction: *how should the robot maintain a global scene estimate from partial observations?* and *which action can make hidden regions visible?* We address the first with a belief map estimator that integrates

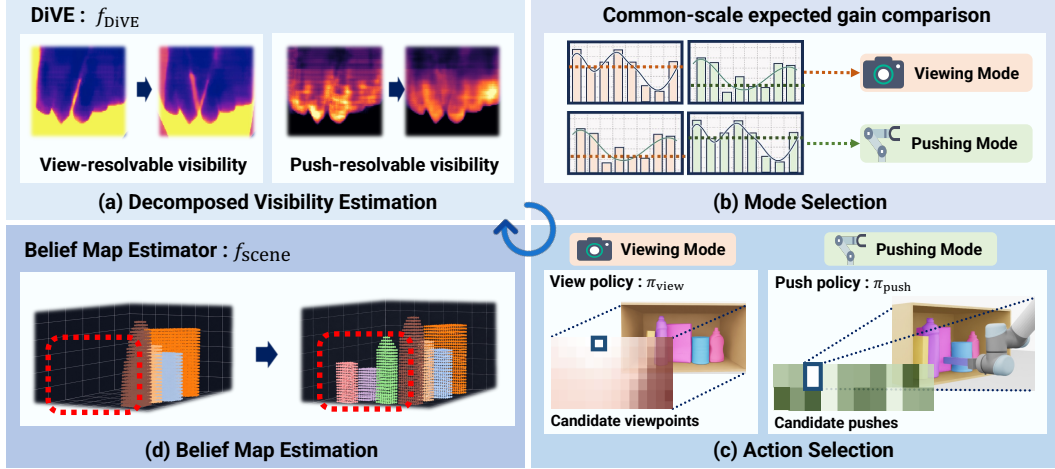


Figure 2: **Overview of the proposed framework.** (a) DiVE (f_{DiVE}) predicts view-resolvable (ϕ_{view}) and push-resolvable (ϕ_{push}) visibility maps from the current partial observation. (b) The maps are converted to a common scale of expected visibility change to select the action mode. (c) The corresponding policy (π_{view} or π_{push}) selects the best viewpoint or push. (d) The belief map estimator (f_{scene}) fuses the new observation and, after a push, the swept region to update the global scene belief ϕ_{scene} .

observations into a global 3D scene belief, and the second with decomposed visibility maps that drive hierarchical view-and-push action selection. As shown in Fig. 2, the framework consists of three components: DiVE, mode-and-action selection, and a belief map estimator.

4.2 Decomposed Visibility Estimator (DiVE)

The role of DiVE is to convert a partial observation into an action-resolvable visibility representation. A unified map of unresolved regions does not specify whether each region is resolvable by viewing or by physical interaction. We therefore decompose visibility into two modality-specific maps: ϕ_{view} , which captures regions whose visibility depends on viewpoint selection, and ϕ_{push} , which captures regions whose visibility can change after pushing. This decomposition preserves the action modality responsible for resolving each region, providing the signal needed for comparing viewing and pushing in Sec. 4.3.

We implement DiVE as a recursive visibility estimator f_{DiVE} with a shared backbone and two output heads. At each time step t , the estimator updates both visibility maps from the current observation o_t and the previous visibility maps:

$$(\phi_{\text{view},t}, \phi_{\text{push},t}) = f_{\text{DiVE}}(\phi_{\text{view},t-1}, \phi_{\text{push},t-1}, o_t). \quad (1)$$

The two heads share the same architecture but are supervised with different targets, reflecting the different mechanisms by which viewpoint motion and pushing change visibility.

View-resolvable visibility (ϕ_{view}). The view head estimates which regions can be resolved through additional viewpoints. For each voxel $v \in \mathcal{V}$, it models how often that voxel is visible from the viewpoints visited since the most recent push. Let $m_k \in \{0, 1\}^{|\mathcal{V}|}$ denote the binary visibility map obtained from camera k on the current scene, and let $\mathcal{K}_t$ denote the set of viewpoints visited since the most recent push, or since the episode start if no push has occurred. We define the training target as the empirical visibility frequency over $\mathcal{K}_t$:

$$\phi_{\text{view},t}^*(v) = \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} m_k(v). \quad (2)$$

Intuitively, ϕ_{view} represents how frequently each voxel is visible across visited viewpoints: high values correspond to voxels visible from many views, and low values to voxels rarely visible from the current view set. Thus, ϕ_{view} serves as a visibility state, rather than a direct gain map, from which the expected gain of an additional viewpoint is derived in Sec. 4.3.

Push-resolvable visibility (ϕ_{push}). The push head estimates which regions can be resolved through physical interaction. Unlike viewpoint changes, pushing alters the scene geometry; a useful push is therefore one that changes the marginal visibility of voxels across the candidate viewpoint set. For a push p , let $\bar{m}^{\text{before}}(v) = \mathbb{E}_{k \sim \mathcal{C}}[m_k^{\text{before}}(v)]$ and $\bar{m}_p^{\text{after}}(v) = \mathbb{E}_{k \sim \mathcal{C}}[m_{k,p}^{\text{after}}(v)]$ denote the marginal visibility of voxel v over candidate viewpoints before and after executing p , respectively. We define the training target as the expected absolute change in marginal visibility induced by a randomly sampled candidate push:

$$\phi_{\text{push},t}^*(v) = \mathbb{E}_{p \sim \mathcal{P}_t} \left[|\bar{m}_p^{\text{after}}(v) - \bar{m}^{\text{before}}(v)| \right], \quad (3)$$

where $\mathcal{P}_t$ and $\mathcal{C}$ denote the candidate push and viewpoint sets, respectively. For supervision generation, every push candidate is executed from the same restored pre-push configuration, and all 98 candidate viewpoints are re-rendered after execution; full details are provided in the Appendix. Unlike ϕ_{view} , which represents a visibility state, ϕ_{push} directly represents an expected visibility change induced by pushing. This target assigns high values to voxels whose visibility is likely to change after pushing, capturing regions that are not merely unobserved but physically resolvable through interaction.

4.3 Mode and Action Selection

Given the decomposed visibility maps from f_{DIVE} , we select an action in two stages. First, we select the action mode by comparing the expected visibility gains of viewing and pushing. Then, conditioned on the selected mode, a mode-specific policy selects the best candidate viewpoint or push. This hierarchical design avoids directly ranking heterogeneous view and push candidates against one another, which can be unstable because their predicted gains arise from different action effects. By comparing modes before ranking candidates, the planner becomes less sensitive to noisy or outlier candidate scores.

Mode selection. The main challenge in mode selection is that the two visibility maps have different meanings. The view map $\phi_{\text{view},t}$ is a per-voxel visibility state accumulated over viewpoints, whereas the push map $\phi_{\text{push},t}$ already represents an expected visibility change induced by pushing. To compare viewing and pushing, we convert both quantities into the same unit: the expected per-voxel visibility change induced by one action of the corresponding mode. We use the absolute change in visibility to measure how much an action changes the visibility state, regardless of whether the change corresponds to newly revealed or newly occluded regions.

For viewing, $\phi_{\text{view},t}$ is a running average over the viewpoints visited so far. Thus, the effect of one additional viewpoint can be measured by how much the updated visibility state $\phi_{\text{view},t+1}$ differs from $\phi_{\text{view},t}$. Since the true distribution of visibility over unvisited viewpoints is not available at test time, we use $\phi_{\text{view},t}(v)$ as a plug-in marginal estimate of the Bernoulli mean for the binary visibility $m_{k_{t+1}}(v)$ observed from a newly selected viewpoint $k_{t+1} \sim \mathcal{C}$. Under this plug-in model, the expected absolute update for voxel v is

$$\mathbb{E}_{k_{t+1} \sim \mathcal{C}} [|\phi_{\text{view},t+1}(v) - \phi_{\text{view},t}(v)|] = \frac{2\phi_{\text{view},t}(v)(1 - \phi_{\text{view},t}(v))}{|\mathcal{K}_t| + 1}, \quad (4)$$

with the derivation and empirical validation provided in the Appendix. Empirically, the approximation shows strong agreement after a few accumulated viewpoints, with normalized mean absolute error (nMAE) below 10% for $|\mathcal{K}_t| \geq 5$ across both low- and high-occlusion scenes. We use this quantity as a mode-selection proxy: it is largest when the current visibility state is uncertain across viewpoints and decreases as a voxel becomes consistently visible or consistently hidden.

The push map $\phi_{\text{push},t}$ is already trained to predict the expected absolute change in marginal visibility caused by pushing. Therefore, after Eq. 4, both viewing and pushing are expressed as expected per-voxel visibility changes. The resulting mode-level gains are

$$\mathcal{G}_{\text{view}}(\phi_{\text{view},t}) = \sum_{v \in \mathcal{V}} \frac{2\phi_{\text{view},t}(v)(1 - \phi_{\text{view},t}(v))}{|\mathcal{K}_t| + 1}, \quad \mathcal{G}_{\text{push}}(\phi_{\text{push},t}) = \sum_{v \in \mathcal{V}} \phi_{\text{push},t}(v). \quad (5)$$

The mode with the higher expected gain is selected as $\mathcal{M}_t = \arg \max_{m \in \{\text{view}, \text{push}\}} \mathcal{G}_m$. Because both gains measure the expected visibility change from a single action, the mode decision is derived directly from the decomposed visibility maps rather than from a hand-tuned switching rule. To avoid applying this comparison to a severely under-observed visibility state, we require at least two accumulated views at the start of an episode and after each push; until then, the planner selects the viewing mode.

Action selection. Once the mode $\mathcal{M}_t$ is selected, we choose the specific action within the selected mode using the corresponding policy, π_{view} or π_{push} . These policies replace online one-step lookahead at test time with learned candidate-ranking functions [41, 42]. Since the mode has already been selected, each policy only needs to compare candidates within a homogeneous action set, making the ranking problem simpler and avoiding cross-mode score calibration.

The view selection policy π_{view} takes the current view-resolvable visibility map $\phi_{\text{view},t}$ and the set of visited viewpoints $\mathcal{K}_t$ as input, and predicts a gain score for each candidate viewpoint $k \in \mathcal{C}$:

$$\pi_{\text{view}} : (\phi_{\text{view},t}, \mathcal{K}_t) \mapsto \{\hat{G}_{\text{view}}(k)\}_{k \in \mathcal{C}}, \quad \hat{G}_{\text{view}}(k) \approx \frac{g_{\text{view}}(k) - \mu_{\text{view}}}{\sigma_{\text{view}}}, \quad (6)$$

where $g_{\text{view}}(k) = \sum_{v \in \mathcal{V}} |\phi_{\text{view},t+1}(v; k) - \phi_{\text{view},t}(v)|$ is the one-step visibility gain of viewpoint k , obtained by updating ϕ_{view} with the pretrained view head of f_{DiVE} after observing from viewpoint k . The mean μ_{view} and standard deviation σ_{view} are computed over all candidate viewpoints in $\mathcal{C}$. This normalization makes the policy predict relative candidate quality within the current viewpoint set, rather than requiring calibrated absolute gain values.

The push selection policy π_{push} similarly ranks candidate pushes within $\mathcal{P}_t$. Each push candidate p is represented by its swept volume, which indicates the region directly affected by the interaction. Given the push-resolvable visibility map $\phi_{\text{push},t}$ and the candidate push set $\mathcal{P}_t$, the policy predicts

$$\pi_{\text{push}} : (\phi_{\text{push},t}, \mathcal{P}_t) \mapsto \{\hat{G}_{\text{push}}(p)\}_{p \in \mathcal{P}_t}, \quad \hat{G}_{\text{push}}(p) \approx \frac{g_{\text{push}}(p) - \mu_{\text{push}}}{\sigma_{\text{push}}}, \quad (7)$$

where $g_{\text{push}}(p) = \sum_{v \in \mathcal{V}} |\bar{m}_p^{\text{after}}(v) - \bar{m}_p^{\text{before}}(v)|$ is the one-step visibility gain of push p . Here, $\bar{m}_p^{\text{before}}$ and $\bar{m}_p^{\text{after}}$ denote the marginal visibility maps before and after executing p , as defined in Sec. 4.2. The mean μ_{push} and standard deviation σ_{push} are computed over the candidate push set $\mathcal{P}_t$.

The next action is selected by maximizing the predicted gain score within the selected mode. At test time, the policies select actions with a single forward pass over the candidate set, avoiding repeated simulation or visibility recomputation for every candidate.

4.4 Belief Map Estimator

While DiVE guides action selection, the belief map estimator maintains the geometric and semantic reconstruction. For geometry, each voxel occupancy is represented by a Beta distribution parameterized by (α_t, β_t) , whose mean $\alpha_t/(\alpha_t + \beta_t)$ gives a bounded occupancy estimate while separating occupied and free-space evidence [9, 13, 43, 44]. For semantics, z_t denotes a per-class voxel field, and the scene belief is $\phi_{\text{scene},t} = (\alpha_t, \beta_t, z_t)$. At each time step, f_{scene} updates this belief from $\phi_{\text{scene},t-1}$, the current observation o_t , and, when applicable, the swept volume Ω_t :

$$\phi_{\text{scene},t} = f_{\text{scene}}(\phi_{\text{scene},t-1}, o_t, \Omega_t). \quad (8)$$

By separating scene belief from decomposed visibility, the framework uses visibility for efficient action selection while retaining a dedicated representation for reconstruction.

5 Experiments

5.1 Experimental Setup

We evaluate DiVE in two settings: *simulation*, using a UR5 in Isaac Sim [45] in front of a cluttered shelf with YCB objects [46], and *real-world*, using a Piper AgileX with a comparable shelf and objects from the same categories, transferred zero-shot. We test on three levels of occlusion—*low*, *high*, and *extreme*—with the model trained only on low- and high-occlusion scenes; extreme

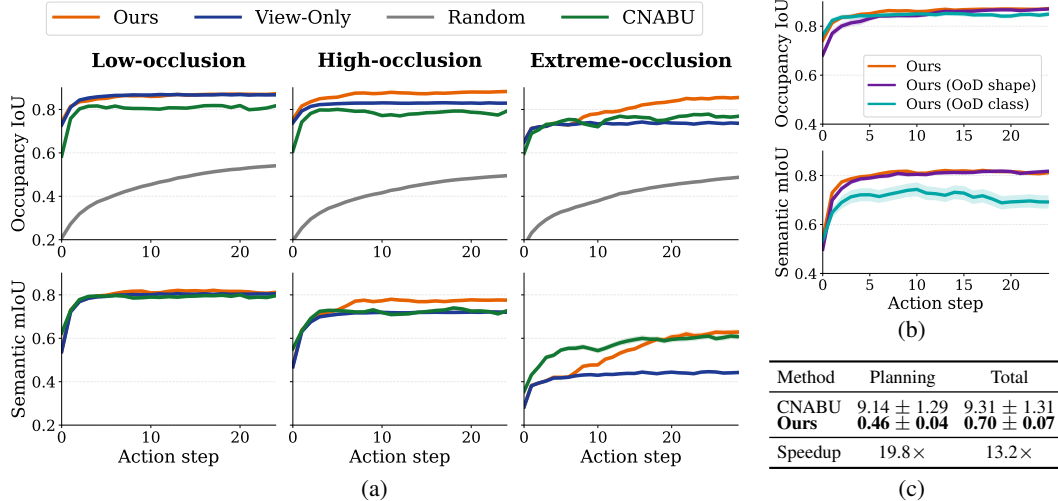


Figure 3: **Main results, generalization, and efficiency.** (a) Reconstruction quality vs. action budget on low-, high-, and extreme-occlusion scenes, comparing DiVE against CNABU [9], *View-Only*, and *Random*. (b) Generalization to objects with out-of-distribution shapes or classes. (c) Per-step compute over 100 episodes. *Planning* is the time to select the next action; *Total* additionally includes belief update and evaluation.

occlusion is held out for evaluation. Full details on scenes, data, training, and metrics are provided in the Appendix.

5.2 Main Comparison

We compare DiVE (Ours) against three baselines. **CNABU** [9] selects view and push actions by evaluating volumetric information gain from predicted post-action beliefs for all candidate actions. We train CNABU end-to-end on our dataset and evaluate it with the same action budget, candidate viewpoints, and candidate pushes as DiVE for a fair comparison. **View-Only** is an ablation of DiVE that always selects the viewing mode while retaining π_{view} , isolating the role of push-resolvable visibility. **Random** uniformly samples viewpoints and integrates observations with a classical occupancy update [47]; it is included only as a lower-bound exploration baseline rather than as an action-selection ablation.

Figure 3(a) reports occupancy IoU and semantic mIoU over the action budget. In low-occlusion scenes, all informed methods perform similarly because most hidden regions can be recovered by viewpoint changes alone. As occlusion increases, *View-Only* and *Random* plateau early, while push-enabled methods continue improving, confirming that some regions are not view-resolvable and require physical interaction. DiVE outperforms CNABU most clearly under high and extreme occlusion. CNABU computes volumetric information gain for all candidate actions from their predicted post-action beliefs, which becomes sensitive to belief prediction errors when large portions of the scene are unobserved. In contrast, DiVE selects actions from decomposed visibility maps, asking whether hidden regions are view-resolvable or push-resolvable rather than predicting the full future belief. This visibility-centered decision becomes more useful as occlusion increases, allowing DiVE to maintain its advantage even under extreme occlusion, an occlusion regime not seen during training. In extreme occlusion, DiVE initially trails CNABU but surpasses it as actions accumulate. This behavior is consistent with its hierarchical selection: rather than switching to pushing based on a single promising push candidate, DiVE compares mode-level expected gains and can continue gathering view information while viewing remains informative.

DiVE is also substantially faster than CNABU. As shown in Fig. 3(c), DiVE reduces per-step planning time by $19.8\times$ and total per-step time by $13.2\times$. This speedup comes from replacing CNABU’s exhaustive candidate-wise computation of post-action belief gains with a mode-specific policy that ranks candidates within the selected mode using a single forward pass.

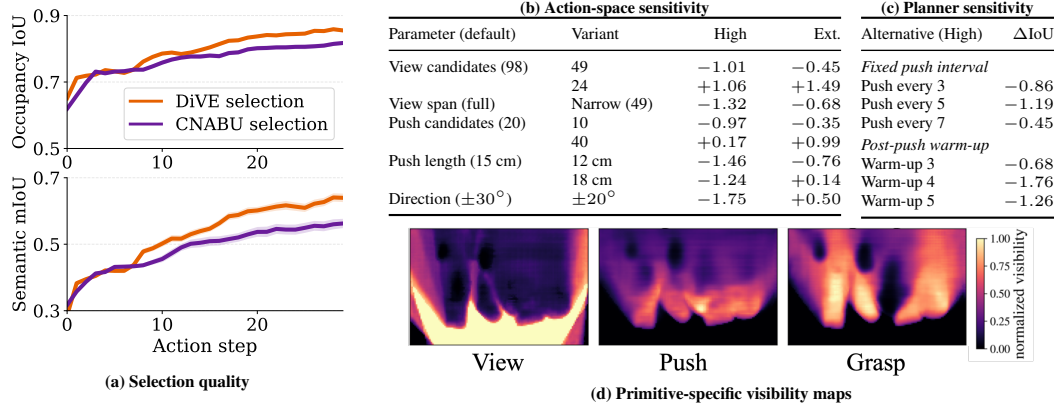


Figure 4: **Action-selection analysis, robustness, and extensibility.** (a) Reconstruction quality with the belief estimator fixed and only the selection strategy changed. (b) Final-IoU change (percentage points) when varying one action-space parameter from the parenthesized default. (c) Final-IoU change under fixed push schedules or longer warm-ups; the default uses learned mode selection and a two-view warm-up. (d) View-, push-, and grasp-resolvable visibility maps; grasp is evaluated only as representation-level feasibility.

5.3 Action-Selection Analysis

Selection quality. To isolate the effect of action selection, we fix the belief estimator f_{scene} and compare decomposed-visibility selection with belief-gain selection (Fig. 4(a)). Since both variants use the same belief estimator, the performance gap reflects action-selection quality rather than reconstruction-module differences. DiVE achieves higher reconstruction performance, showing that decomposed visibility leads to more informative action choices than the belief-gain criterion.

Visibility decomposition. DiVE represents visibility using two separate maps, ϕ_{view} and ϕ_{push} , rather than a single unified map. The unified baseline combines estimated view-resolvable visibility change and push-resolvable gain in one channel; implementation details are provided in the Appendix. For each candidate push with swept volume Ω_p , we compute a map-based score $s(p) = \sum_{v \in \Omega_p} \phi(v)$ using either ϕ_{push} or the unified map, and correlate this score with the true push gain over 76,721 policy-training samples. Scores from ϕ_{push} correlate more strongly with true push gain than scores from the unified map (0.436 vs. 0.204). The full distribution is shown in the Appendix. This shows that decomposing visibility preserves the push-specific cue needed for candidate ranking, whereas mixing view- and push-resolvable signals dilutes this cue and weakens the subsequent view-push decision.

5.4 Robustness and Additional-Primitive Feasibility

Action-space and planner sensitivity. Figure 4(b-c) varies one setting at a time from the default action space and two-view warm-up. Across action-space variants, $|\Delta \text{IoU}| \leq 1.75$ percentage points. Fixed schedules reduce high-occlusion IoU by 0.45–1.19 points, while longer warm-ups provide no improvement, supporting learned gain-based mode selection.

Push execution and risk. Using the success criterion detailed in the Appendix, 180 of 181 simulation pushes succeed (99.4%), as do all pushes executed in the 10 real-world episodes. Because failures are rare under the constrained primitive, the current planner does not model execution probability; future work could weight visibility gain by push success probability.

Additional-primitive feasibility. As a representation-level proof of concept, we train a grasp-resolvable head from the marginal visibility change caused by removing a grasped object. The learned map localizes grasp-resolvable regions (Fig. 4(d)), but this experiment does not evaluate closed-loop grasp selection, joint view-push-grasp episodes, or downstream manipulation.

5.5 Generalization

We evaluate whether DiVE remains effective under two object distribution shifts (Fig. 3(b)). For *shape*, we rescale every object in the low-occlusion test set by a random factor in $[0.8, 1.2]$ and evaluate zero-shot. For *category*, we introduce two unseen object categories and lightly finetune only the belief estimator on 30 low-occlusion scenes, keeping the visibility and action-selection modules frozen. DiVE maintains similar reconstruction quality across both shifts. These results suggest that DiVE generalizes zero-shot to object shape variation, while unseen categories can be handled with lightweight adaptation of the reconstruction module rather than retraining the full framework.

5.6 Real-World Experiments

We deploy DiVE zero-shot on the Piper AgileX setup described in Sec. 5.1. We construct 10 real cluttered shelf scenes using YCB-category objects whose shapes and materials differ from simulation, and compare against View-Only over a 20-action budget. We report object-level discovery metrics: correctly found (CF), misclassified but found (MBF), missed, and initially occluded object discovery (Occ. Disc.). The real-world setup and metrics are detailed in the Appendix.

Table 1 shows that DiVE improves reconstruction after zero-shot transfer to real shelves. It increases correctly found objects from 54.3% to 65.5% and reduces missed objects from 45.7% to 29.1%. The largest gain is in initially occluded objects, where DiVE discovers 79.2% compared with 54.2% for View-Only, suggesting that its learned action selection transfers to real scenes by choosing interactions that expose regions unreachable by viewpoint changes alone. The nonzero MBF rate reflects cases where pushing exposes objects that are geometrically recovered but semantically mislabeled; View-Only instead leaves many of these regions empty, so they are counted as missed. Overall, these results demonstrate the real-world transferability of visibility-based view-and-push exploration.

Method	CF $\uparrow$	MBF	Missed $\downarrow$	Occ. Disc. $\uparrow$
View-only	54.3 $\pm$ 10.2	0.0 $\pm$ 0.0	45.7 $\pm$ 10.2	54.2 $\pm$ 35.4
Ours	65.5$\pm$14.2	5.4 $\pm$ 6.9	29.1$\pm$15.4	79.2$\pm$23.3
Δ	+11.2	+5.4	-16.6	+25.0

Table 1: Zero-shot real-world results over 10 scenes. All values are percentages.

6 Conclusion

We presented DiVE, a modality-conditioned visibility representation that decomposes hidden regions into view- and push-resolvable components. This representation lets an established hierarchical planner compare modes without candidate-wise post-action belief prediction or a hand-tuned cross-mode threshold. Experiments in simulation and real cluttered shelves show more informative action selection and lower planning cost, with the largest gains under severe occlusion.

Limitations and future work. Evaluation is limited to one cluttered-shelf domain, discrete action candidates, and no explicit execution-cost or risk model. The grasp-resolvable head in Sec. 5.4 is only a representation-level feasibility demonstration, not closed-loop or downstream-task validation. Future work should study risk-aware gain, diverse layouts and interaction primitives, open-vocabulary semantics, and downstream manipulation.

Acknowledgments

The authors thank the anonymous reviewers and the Area Chair for their constructive feedback, which helped improve the clarity of the paper and motivated additional analyses of action-space sensitivity, push reliability, and extensibility to additional manipulation primitives. We also thank our colleagues for their helpful discussions and feedback.

References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Z. Fu, T. Z. Zhao, and C. Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.
- [6] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. M. M. Shafiullah, and L. Pinto. Ok-robot: What really matters in integrating open-knowledge models for robotics. *arXiv preprint arXiv:2401.12202*, 2024.
- [7] A. Rosinol, M. Abate, Y. Chang, and L. Carlone. Kimera: an open-source library for real-time metric-semantic localization and mapping. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 1689–1696. IEEE, 2020.
- [8] C. Xie, Y. Xiang, A. Mousavian, and D. Fox. Unseen object instance segmentation for robotic environments. *IEEE Transactions on Robotics*, 37(5):1343–1359, 2021.
- [9] J. M. C. Marques, N. Dengler, T. Zaenker, J. Mücke, S. Wang, M. Bennewitz, and K. Hauser. Map space belief prediction for manipulation-enhanced mapping. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [10] N. Dengler, S. Pan, V. Kalagaturu, R. Menon, M. Dawood, and M. Bennewitz. Viewpoint push planning for mapping of unknown confined spaces. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1178–1184, 2023.
- [11] T. Novkovic, R. Pautrat, F. Furrer, M. Breyer, R. Siegwart, and J. Nieto. Object finding in cluttered scenes using interactive perception. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [12] O. Zenkri, N. A. Vien, and G. Neumann. Hierarchical policy learning for mechanical search. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 1954–1960. IEEE, 2022.
- [13] N. Dengler, J. Mücke, R. Menon, and M. Bennewitz. Efficient manipulation-enhanced semantic mapping with uncertainty-informed action selection. In *Proceedings of the IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2025.

- [14] C. Connolly. The determination of next best views. In *Proceedings. 1985 IEEE international conference on robotics and automation*, volume 2, pages 432–435. IEEE, 1985.
- [15] S. Isler, R. Sabzevari, J. Delmerico, and D. Scaramuzza. An information gain formulation for active volumetric 3d reconstruction. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3477–3484, 2016.
- [16] J. Delmerico, S. Isler, R. Sabzevari, and D. Scaramuzza. A comparison of volumetric information gain metrics for active 3d object reconstruction. *Autonomous Robots*, 42(2):197–208, 2018.
- [17] J. Daudelin and M. Campbell. An adaptable, probabilistic, next-best view algorithm for reconstruction of unknown 3-d objects. *IEEE Robotics and Automation Letters*, 2(3):1540–1547, 2017.
- [18] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart. Receding horizon” next-best-view” planner for 3d exploration. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1462–1468. IEEE, 2016.
- [19] Z. Jia, Y. Li, Q. Hao, and S. Zhang. Pb-nbv: Efficient projection-based next-best-view planning framework for reconstruction of unknown objects. *IEEE Robotics and Automation Letters*, 2025. doi:[10.1109/LRA.2025.3573631](https://doi.org/10.1109/LRA.2025.3573631).
- [20] X. Chen, Q. Li, T. Wang, T. Xue, and J. Pang. Gennbv: Generalizable next-best-view policy for active 3d reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16436–16445, 2024.
- [21] D. Peralta, J. Casimiro, A. M. Nilles, J. A. Aguilar, R. Atienza, and R. Cajote. Next-best view policy for 3d reconstruction. In *European Conference on Computer Vision*, pages 558–573. Springer, 2020.
- [22] L. Jin, X. Chen, J. Rückin, and M. Popović. Neu-nbv: Next best view planning using uncertainty estimation in image-based neural rendering. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11305–11312. IEEE, 2023.
- [23] H. Ren and A. H. Qureshi. Robot active neural sensing and planning in unknown cluttered environments. *IEEE Transactions on Robotics*, 39(4):2738–2750, 2023.
- [24] Z. Feng, H. Zhan, Z. Chen, Q. Yan, X. Xu, C. Cai, B. Li, Q. Zhu, and Y. Xu. Naruto: Neural active reconstruction from uncertain target observations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21572–21583, 2024.
- [25] W. Jiang, B. Lei, and K. Daniilidis. Fisherrf: Active view selection and mapping with radiance fields using fisher information. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XIII*, pages 422–440, 2024. doi:[10.1007/978-3-031-72624-8_24](https://doi.org/10.1007/978-3-031-72624-8_24).
- [26] S. Xue, J. Dill, P. Mathur, F. Dellaert, P. Tsotra, and D. Xu. Neural visibility field for uncertainty-driven active mapping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18122–18132, 2024.
- [27] J. Bohg, K. Hausman, B. Sankaran, O. Brock, D. Kragic, S. Schaal, and G. S. Sukhatme. Interactive perception: Leveraging action in perception and perception in action. *IEEE Transactions on Robotics*, 33(6):1273–1291, 2017.
- [28] D. Katz and O. Brock. Manipulating articulated objects with interactive perception. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 272–277, 2008.

- [29] R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.
- [30] R. Bajcsy, Y. Aloimonos, and J. K. Tsotsos. Revisiting active perception. *Autonomous Robots*, 42(2):177–196, 2018.
- [31] S. Chen, Y. Li, and N. M. Kwok. Active vision in robotic systems: A survey of recent developments. *The International Journal of Robotics Research*, 30(11):1343–1377, 2011.
- [32] A. Eitel, N. Hauff, and W. Burgard. Self-supervised transfer learning for instance segmentation through physical interaction. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4020–4026. IEEE, 2019.
- [33] B. Serhan, H. Pandya, A. Kucukyilmaz, and G. Neumann. Push-to-see: Learning non-prehensile manipulation to enhance instance segmentation via deep q-learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [34] H. H. Qian, Y. Lu, K. Ren, G. Wang, N. Khargonkar, Y. Xiang, and K. Hang. Riseg: Robot interactive object segmentation via body frame-invariant features. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [35] M. Danielczuk, A. Kurenkov, A. Balakrishna, M. Matl, D. Wang, R. Martín-Martín, A. Garg, S. Savarese, and K. Goldberg. Mechanical search: Multi-step retrieval of a target object occluded by clutter. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1614–1621, 2019.
- [36] A. Kurenkov, J. Taglic, R. Kulkarni, M. Dominguez-Kuhne, A. Garg, R. Martín-Martín, and S. Savarese. Visuomotor mechanical search: Learning to retrieve target objects in clutter. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8408–8414. IEEE, 2020.
- [37] M. Danielczuk, A. Angelova, V. Vanhoucke, and K. Goldberg. X-ray: Mechanical search for an occluded object by minimizing support of learned occupancy distributions. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9577–9584. IEEE, 2020.
- [38] B. Huang, S. D. Han, J. Yu, and A. Boularias. Visual foresight trees for object retrieval from clutter with nonprehensile rearrangement. *IEEE Robotics and Automation Letters*, 7(1):231–238, 2021.
- [39] H. Moravec and A. Elfes. High resolution maps from wide angle sonar. In *Proceedings. 1985 IEEE international conference on robotics and automation*, volume 2, pages 116–121. IEEE, 1985.
- [40] B. Curless and M. Levoy. A volumetric method for building complex models from range images. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*, pages 303–312, 1996.
- [41] N. Frahm, D. Zhao, A. D. Beltran, R. Alterovitz, J.-M. Frahm, J. Oliva, and R. Sengupta. Vin-nbv: A view introspection network for next-best-view selection. *arXiv preprint arXiv:2505.06219*, 2025.
- [42] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [43] A. Asgharivaskasi and N. Atanasov. Active bayesian multi-class mapping from range and semantic segmentation observations. In *2021 IEEE international conference on robotics and automation (ICRA)*, pages 1–7. IEEE, 2021.

- [44] J.-L. Matez-Bandera, P. Ojeda, J. Monroy, J. Gonzalez-Jimenez, and J.-R. Ruiz-Sarmiento. Voxeland: Probabilistic instance-aware semantic mapping with evidence-based uncertainty quantification. *arXiv preprint arXiv:2411.08727*, 2024.
- [45] NVIDIA. Isaac sim. <https://developer.nvidia.com/isaac/sim>, 2025. Version 5.1.0.
- [46] B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar. Benchmarking in manipulation research: The YCB object and model set and benchmarking protocols. *IEEE Robotics & Automation Magazine*, 22(3):36–52, 2015. doi:10.1109/MRA.2015.2448951.
- [47] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. Intelligent Robotics and Autonomous Agents. MIT Press, Cambridge, MA, USA, 2005.

A Methodological Details

A.1 Algorithm

Algorithm 1 Active Scene Reconstruction with Decomposed Visibility

```

1: Initialize  $\phi_{\text{view}, -1}, \phi_{\text{push}, -1} \leftarrow \mathbf{0}$ 
2: Initialize  $\phi_{\text{scene}, -1} \leftarrow \mathbf{0.5}$  and  $\Omega_0 \leftarrow \mathbf{0}$ 
3: Acquire initial observation  $o_0$  from viewpoint  $k_0$ 
4: Set  $\mathcal{K}_0 \leftarrow \{k_0\}$   $\triangleright$  viewpoints visited since the most recent push
5: for  $t = 0, 1, \dots, T - 1$  do
6:   // (i) Visibility and belief update
7:    $(\phi_{\text{view}, t}, \phi_{\text{push}, t}) \leftarrow f_{\text{DiVE}}(\phi_{\text{view}, t-1}, \phi_{\text{push}, t-1}, o_t)$ 
8:    $\phi_{\text{scene}, t} \leftarrow f_{\text{scene}}(\phi_{\text{scene}, t-1}, o_t, \Omega_t)$ 
9:   // (ii) Mode selection
10:   $\mathcal{G}_{\text{view}} \leftarrow \sum_{v \in \mathcal{V}} \frac{2\phi_{\text{view}, t}(v)(1 - \phi_{\text{view}, t}(v))}{|\mathcal{K}_t| + 1}$ 
11:   $\mathcal{G}_{\text{push}} \leftarrow \sum_{v \in \mathcal{V}} \phi_{\text{push}, t}(v)$ 
12:  if  $|\mathcal{K}_t| < 2$  then
13:     $\mathcal{M}_t \leftarrow \text{view}$   $\triangleright$  two-view warm-up
14:  else
15:     $\mathcal{M}_t \leftarrow \arg \max_{m \in \{\text{view}, \text{push}\}} \mathcal{G}_m$ 
16:  end if
17:  // (iii) Action selection
18:  if  $\mathcal{M}_t = \text{view}$  then
19:     $a_t \leftarrow \pi_{\text{view}}(\phi_{\text{view}, t}, \mathcal{K}_t)$ 
20:    Execute viewpoint action  $a_t$  and acquire  $o_{t+1}$  from viewpoint  $k_{t+1} = a_t$ 
21:     $\Omega_{t+1} \leftarrow \mathbf{0}$ 
22:     $\mathcal{K}_{t+1} \leftarrow \mathcal{K}_t \cup \{k_{t+1}\}$ 
23:  else
24:     $a_t \leftarrow \pi_{\text{push}}(\phi_{\text{push}, t}, \mathcal{P}_t)$ 
25:    Execute push action  $a_t$  and acquire  $o_{t+1}$  from viewpoint  $k_{t+1}$ 
26:     $\Omega_{t+1} \leftarrow \Omega(a_t)$ 
27:     $\mathcal{K}_{t+1} \leftarrow \{k_{t+1}\}$   $\triangleright$  reset after the scene geometry changes
28:  end if
29: end for
30: return  $\phi_{\text{scene}, T}$ 

```

A.2 Derivation and Empirical Validation of the View-Mode Gain Proxy

We derive the analytic view-mode gain proxy used in Eq. 4 and empirically validate whether it matches the data-derived visibility change from actual candidate viewpoints. This validation is important because the planner compares viewing and pushing on a common expected-visibility-change scale. The view proxy therefore does not need to predict the exact effect of every candidate viewpoint, but it should provide a stable estimate of whether additional viewing is still expected to change the visibility state.

Setup. Fix a voxel $v \in \mathcal{V}$. Let $\mathcal{K}_t$ denote the set of viewpoints visited since the most recent push, and let $n := |\mathcal{K}_t|$. The reset after a push is necessary because pushing changes the scene geometry; visibility statistics accumulated before the push no longer describe the same physical arrangement.

Let $m_k(v) \in \{0, 1\}$ denote the binary visibility of voxel v from viewpoint k . The view-resolvable visibility state is defined as the running average of the binary visibility maps over the accumulated viewpoints:

$$\phi_{\text{view}, t}(v) = \frac{1}{n} \sum_{k \in \mathcal{K}_t} m_k(v). \quad (9)$$

Let k' denote one additional viewpoint to be observed. We define

$$X := m_{k'}(v) \in \{0, 1\} \quad (10)$$

as the Bernoulli random variable indicating whether voxel v is visible from this additional viewpoint. After incorporating this additional viewpoint, the updated running average becomes

$$\phi_{\text{view},t+1}(v) = \frac{1}{n+1} \left(\sum_{k \in \mathcal{K}_t} m_k(v) + X \right). \quad (11)$$

Derivation. Let

$$S := \sum_{k \in \mathcal{K}_t} m_k(v). \quad (12)$$

Subtracting Eq. 9 from Eq. 11, we obtain

$$\phi_{\text{view},t+1}(v) - \phi_{\text{view},t}(v) = \frac{S+X}{n+1} - \frac{S}{n} \quad (13)$$

$$= \frac{n(S+X) - (n+1)S}{n(n+1)} \quad (14)$$

$$= \frac{nX - S}{n(n+1)}. \quad (15)$$

Since $S = n \phi_{\text{view},t}(v)$,

$$\phi_{\text{view},t+1}(v) - \phi_{\text{view},t}(v) = \frac{nX - n \phi_{\text{view},t}(v)}{n(n+1)} \quad (16)$$

$$= \frac{X - \phi_{\text{view},t}(v)}{n+1}. \quad (17)$$

Taking the absolute value gives

$$|\phi_{\text{view},t+1}(v) - \phi_{\text{view},t}(v)| = \frac{|X - \phi_{\text{view},t}(v)|}{n+1}. \quad (18)$$

We next use $\phi_{\text{view},t}(v)$ as a plug-in estimate of the Bernoulli mean for the visibility of voxel v from an additional viewpoint:

$$\Pr(X = 1) = \phi_{\text{view},t}(v), \quad \Pr(X = 0) = 1 - \phi_{\text{view},t}(v). \quad (19)$$

Writing $p := \phi_{\text{view},t}(v)$ for brevity,

$$\mathbb{E}[|X - p|] = \Pr(X = 1)|1 - p| + \Pr(X = 0)|0 - p| \quad (20)$$

$$= p(1 - p) + (1 - p)p \quad (21)$$

$$= 2p(1 - p). \quad (22)$$

Substituting this into Eq. 18, we obtain

$$\mathbb{E}_{k'}[|\phi_{\text{view},t+1}(v) - \phi_{\text{view},t}(v)|] = \frac{2 \phi_{\text{view},t}(v) (1 - \phi_{\text{view},t}(v))}{n+1} = \frac{2 \phi_{\text{view},t}(v) (1 - \phi_{\text{view},t}(v))}{|\mathcal{K}_t| + 1}. \quad (23)$$

We denote this analytic plug-in estimate as

$$A_n(v) := \frac{2 \phi_{\text{view},t}(v) (1 - \phi_{\text{view},t}(v))}{n+1}. \quad (24)$$

Interpretation. The term $2p(1 - p)$ arises from the expected absolute deviation of a Bernoulli variable from its mean, and is equal to twice the variance of a Bernoulli variable with mean p . It is maximized at $p = 0.5$ and vanishes as p approaches either 0 or 1. Therefore, the proxy assigns the largest expected update to voxels whose visibility is inconsistent across viewpoints, and assigns small values to voxels that are consistently visible or consistently invisible. The denominator $n + 1$ captures the diminishing influence of one additional viewpoint as more viewpoints are accumulated. Thus, $A_n(v)$ estimates how much one more view is expected to change the current visibility state, not whether the voxel is occupied or semantically correct.

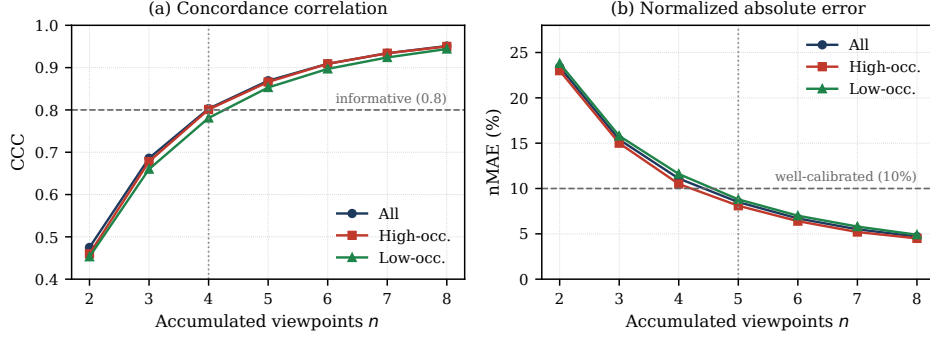


Figure 5: Empirical validation of the analytic view-mode gain proxy. We compare the analytic proxy $\hat{G}_{\text{view}}^{(n)}$ with the data-derived empirical visibility-state change $G_{\text{view}}^{(n),\text{emp}}$ as the number of accumulated viewpoints n increases. The proxy is noisy for the first few observations, but becomes informative around $n = 4$ and well calibrated from $n = 5$ onward. The same trend appears in both low- and high-occlusion scenes, indicating that the approximation is not specific to one occlusion regime.

Empirical validation protocol. The derivation above uses a plug-in Bernoulli model. To test whether this approximation is useful on our data, we compare the analytic proxy with the empirical visibility-state change computed from actual candidate-view visibility maps.

For each validation instance, we construct a partial viewpoint set $\mathcal{K}_t$ of size n and compute $\phi_{\text{view},t}$ using Eq. 9. We then evaluate the actual effect of adding one more candidate viewpoint using the ground-truth binary visibility maps:

$$\Delta_n^{\text{emp}}(v) = \frac{1}{|\mathcal{C}|} \sum_{k' \in \mathcal{C}} \left| \frac{n \phi_{\text{view},t}(v) + m_{k'}(v)}{n+1} - \phi_{\text{view},t}(v) \right|. \quad (25)$$

This quantity is the data-derived expected absolute update of the running visibility state after one additional viewpoint.

Since mode selection uses the voxel-summed gain rather than individual voxel values, we compare the corresponding map-level quantities:

$$\hat{G}_{\text{view}}^{(n)} = \sum_{v \in \mathcal{V}} A_n(v), \quad G_{\text{view}}^{(n),\text{emp}} = \sum_{v \in \mathcal{V}} \Delta_n^{\text{emp}}(v). \quad (26)$$

We report Pearson correlation, concordance correlation coefficient (CCC), R^2 with respect to the identity line $y = x$, normalized mean absolute error (nMAE), and normalized bias between $\hat{G}_{\text{view}}^{(n)}$ and $G_{\text{view}}^{(n),\text{emp}}$. We evaluate this agreement for $n = 1, \dots, 8$ on all validation samples, and separately on low- and high-occlusion scenes. Figure 5 visualizes the agreement between the analytic proxy and the empirical view-induced visibility change as the number of accumulated viewpoints increases. Table 2 reports representative numerical values from this validation.

Discussion. The plug-in proxy is less reliable when only one or two viewpoints have been accumulated, which is expected because $\phi_{\text{view},t}$ is then estimated from very sparse visibility observations. However, its agreement with the empirical visibility-state change improves rapidly after a few views. Over all validation samples, CCC increases from 0.686 at $n = 3$ to 0.803 at $n = 4$, while nMAE decreases from 15.4% to 11.1%. From $n = 5$ onward, nMAE falls below 10%, and by $n = 8$ the proxy reaches CCC = 0.951 and nMAE = 4.7%.

The same trend holds under both occlusion regimes. At $n = 4$, high- and low-occlusion scenes obtain CCC values of 0.801 and 0.781, respectively. At $n = 5$, nMAE is below 9% in both splits. This indicates that the proxy is not specific to a single visibility regime, but provides a consistent estimate of expected view-induced visibility change across different clutter densities.

Table 2: Empirical validation of the view-mode gain proxy. The proxy becomes informative after a few accumulated viewpoints: at $n = 4$, CCC is around 0.80 with nMAE near 11%, and from $n = 5$ onward nMAE falls below 10% across both low- and high-occlusion scenes. Full trends over $n = 1, \dots, 8$ are shown in Fig. 5.

Split	n	N	Pearson	CCC	$R_{y=x}^2$	nMAE
All	3	12.8k	0.877	0.686	0.212	15.4%
All	4	12.7k	0.920	0.803	0.558	11.1%
All	5	12.8k	0.941	0.869	0.718	8.5%
All	8	12.7k	0.970	0.951	0.901	4.7%
High-occ.	4	6.4k	0.915	0.801	0.548	10.5%
High-occ.	5	6.5k	0.936	0.866	0.710	8.1%
Low-occ.	4	6.4k	0.915	0.781	0.489	11.6%
Low-occ.	5	6.4k	0.937	0.853	0.676	8.8%

These results support using A_n as a mode-selection proxy rather than as an exact per-voxel predictor. The planner compares the voxel-summed view gain $\hat{G}_{\text{view}}^{(n)}$ with the push gain, so individual voxel-level errors are averaged over the scene. Therefore, the proxy only needs to estimate whether additional viewing is still expected to meaningfully change the visibility state at the mode level. Empirically, this condition is reasonably met around $n = 4$ and becomes more reliable from $n = 5$ onward. This matches the intended use of the proxy in the hierarchical planner: viewing is preferred while the accumulated visibility state can still be meaningfully updated, whereas pushing becomes favorable when the expected gain from additional views is limited relative to the predicted push-induced visibility change.

B Implementation Details

B.1 Simulation Data Generation

We generate cluttered shelf scenes in Isaac Sim using a single shelf workspace and objects drawn from the YCB object set. The in-distribution object categories used for training and standard evaluation are listed in Table 3. The model is trained on low- and high-occlusion scenes, while extreme-occlusion scenes are held out for evaluation as described in Sec. C.3.

Table 3: YCB objects used in the in-distribution cluttered shelf scenes.

Index	YCB object
0	003_cracker_box
1	004_sugar_box
2	006_mustard_bottle
3	021_bleach_cleanser
4	002_master_chef_can
5	005_tomato_soup_can
6	009_gelatin_box
7	010_potted_meat_can
8	008_pudding_box
9	007_tuna_fish_can
10	061_foam_brick

Voxel representation. Each scene is discretized into $\mathcal{V} \subset \mathbb{R}^{120 \times 60 \times 80}$, covering a $60 \times 30 \times 40$ cm workspace in front of the shelf. Each voxel is therefore approximately 5 mm on a side. We use the same voxel grid for visibility estimation, action selection, belief-map updating, and evaluation.

Candidate viewpoints. The candidate viewpoint set $\mathcal{C}$ contains 98 camera poses arranged on a 7×14 planar grid in front of the shelf. Each camera is oriented toward the center of the shelf so that

the candidate set covers different viewing angles over the same workspace. We index viewpoints as

$$\text{cam_idx} = \text{row} \times 14 + \text{col}. \quad (27)$$

All methods use the same calibrated RGB-D camera model and the same candidate viewpoint set. Thus, performance differences arise from action selection and belief updating rather than from changes in sensing geometry.

Candidate pushes and swept maps. At each decision step, we construct a candidate push set $\mathcal{P}_t$ with 20 actions. Candidate contacts are sampled from frontier regions near the boundary between observed occupied space and unobserved space. For each contact, we sample a pushing direction within $\pm 30^\circ$ of the shelf depth axis and apply a fixed push length of 15 cm. Each push candidate $p \in \mathcal{P}_t$ is represented by its swept map Ω_p , which marks the voxels physically affected by the push. The swept map is used by the push policy and is passed to the belief map estimator after executing an interaction.

Episode data. During data generation, we render all 98 candidate viewpoints for each scene state. This allows us to compute dense supervision for view-resolvable visibility, push-resolvable visibility, and candidate-level policy targets without repeatedly simulating online planning rollouts. Push sequences contain multiple interaction states, and all samples derived from the same initial scene inherit the same train/validation split to prevent leakage.

Push supervision and execution handling. For every pre-push scene, we store the complete physical configuration and render all 98 candidate viewpoints to obtain the pre-push marginal visibility. Before evaluating each push candidate, we restore this identical configuration, execute the candidate, and re-render all 98 viewpoints to compute its post-push marginal visibility. Thus, candidate targets differ only in the effect of the evaluated push rather than in the scene state from which it starts. Invalid candidates are masked during training and inference. After all candidates have been evaluated, one valid post-push configuration is sampled as the next state when generating multi-push sequences.

We define an executed push as successful when it contacts the intended object without tipping it or dropping it from the workspace; failures are detected from the resulting object configuration. Under this criterion, 180 of 181 executed simulation pushes were successful (99.4%), and all pushes executed in the 10 real-world episodes were successful. Because failures were rare under the constrained primitive, the current selector does not explicitly predict success probability; a risk-aware extension could weight predicted visibility gain by execution success probability, cost, or safety.

Feasibility beyond pushing. To test whether the representation can accommodate another interaction primitive, we additionally train a grasp-resolvable visibility head using the marginal visibility change produced by removing a grasped object as supervision. The learned head produces spatially localized grasp-resolvable responses, showing that the modality-conditioned representation can be instantiated for a third primitive. This experiment is a representation-level proof of concept only: it does not evaluate closed-loop grasp selection, joint view–push–grasp episodes, or improvement on a downstream manipulation task.

B.2 Network Architecture

The implementation follows the decomposition introduced in the main paper: DiVE predicts two visibility maps, the mode-specific policies rank candidate actions, and the belief map estimator maintains the global reconstruction. We describe the modules using the paper notation rather than implementation-specific code names.

DiVE. DiVE, denoted by f_{DiVE} , is implemented as a shared volumetric backbone with two output heads. The input consists of the previous view-resolvable visibility state, the previous push-

resolvable visibility state, and the current voxel observation:

$$[\phi_{\text{view},t-1}, \phi_{\text{push},t-1}, o_t^{\text{vox}}]. \quad (28)$$

The network outputs two voxel-wise logits:

$$(\ell_{\text{view},t}, \ell_{\text{push},t}) = f_{\text{DiVE}}(\phi_{\text{view},t-1}, \phi_{\text{push},t-1}, o_t^{\text{vox}}), \quad (29)$$

which are converted to visibility maps by sigmoid activations:

$$\phi_{\text{view},t} = \sigma(\ell_{\text{view},t}), \quad \phi_{\text{push},t} = \sigma(\ell_{\text{push},t}). \quad (30)$$

The view head is supervised with accumulated visibility frequency over visited viewpoints, whereas the push head is supervised with marginal visibility change induced by interaction. The swept map is not an input to DiVE; it is used by the push policy and by the belief map estimator after a push is executed.

View policy. The view policy π_{view} ranks the 98 candidate viewpoints. It takes the current view-resolvable visibility map together with the viewpoint history $\mathcal{K}_t$, encoded as a seen-view mask. The policy outputs one logit per candidate viewpoint:

$$\pi_{\text{view}}(\phi_{\text{view},t}, \mathcal{K}_t) \rightarrow \{\hat{\ell}_{\text{view}}(k)\}_{k \in \mathcal{C}}. \quad (31)$$

At inference time, the next viewpoint is selected from the candidate set using the predicted scores.

Push policy. The push policy π_{push} ranks the 20 candidate pushes. Each candidate is represented by its swept map Ω_p , and invalid push candidates are masked during training and inference. The implementation scores each push using its overlap with the push-resolvable visibility map together with a learned residual term. Candidate-level self-attention is used so that candidate pushes can be compared within the same decision state. The policy outputs one logit per push:

$$\pi_{\text{push}}(\phi_{\text{push},t}, \mathcal{P}_t) \rightarrow \{\hat{\ell}_{\text{push}}(p)\}_{p \in \mathcal{P}_t}. \quad (32)$$

The selected push is the valid candidate with the highest predicted score.

Belief map estimator. The belief map estimator f_{scene} maintains the global geometric and semantic reconstruction. Occupancy is represented by Beta parameters (α_t, β_t) , and semantics are represented by voxel-wise semantic logits. The implementation uses $n_{\text{sem}} = 15$ semantic channels: class 0 denotes free space, 11 object classes are active in the in-distribution data, and three additional slots are reserved for unseen-class finetuning.

The input to the belief map estimator contains the previous occupancy belief, the current per-view occupancy observation, the swept map, the current semantic observation, and the previous semantic belief:

$$[\alpha_{t-1}/50, \beta_{t-1}/50, o_t^{\text{occ}}, \Omega_t, o_t^{\text{sem}}, \phi_{\text{scene},t-1}^{\text{sem}}]. \quad (33)$$

The estimator outputs updated occupancy parameters and semantic logits:

$$(\alpha_t, \beta_t, \hat{s}_t) = f_{\text{scene}}(\phi_{\text{scene},t-1}, o_t, \Omega_t). \quad (34)$$

The semantic belief passed to the next step is computed from the detached softmax of $\hat{s}_t$. This stabilizes sequential belief updates and prevents gradients from propagating through long rollout histories.

B.3 Training Targets and Losses

DiVE. The view head is supervised by the accumulated visibility-frequency target in Eq. 2. For a sequence of visited viewpoints, the target is the running average of per-view binary visibility maps. The push head is supervised by the push-resolvable visibility target in Eq. 3, computed from the marginal visibility change induced by candidate pushes. We train DiVE with soft binary cross entropy:

$$\mathcal{L}_{\text{DiVE}} = \text{BCEWithLogits}(\ell_{\text{view}}, \phi_{\text{view}}^*) + \lambda_{\text{push}} \text{BCEWithLogits}(\ell_{\text{push}}, \phi_{\text{push}}^*), \quad (35)$$

where $\lambda_{\text{push}} = 1.0$.

Table 4: Implementation-level module specification.

Module	Role	Input	Output
f_{DiVE}	Decomposed visibility estimation	Previous ϕ_{view} , previous ϕ_{push} , current voxel observation	Voxel-wise logits for ϕ_{view} and ϕ_{push}
π_{view}	Viewpoint ranking	ϕ_{view} and seen-view mask	98 candidate-view logits
π_{push}	Push ranking	ϕ_{push} , 20 swept maps, valid-candidate mask	20 candidate-push logits
f_{scene}	Belief map estimation	Beta occupancy belief, observation, swept map, semantic belief	Updated Beta occupancy parameters and 15-way semantic logits

View policy. The view policy is trained by distilling candidate-level visibility gains. For each training state, candidate gains $g_{\text{view}}(k)$ are computed by measuring how much the view-resolvable visibility state changes after adding candidate viewpoint k . Since the absolute gain scale varies across scenes, we normalize gains within each candidate set:

$$\tilde{g}_{\text{view}}(k) = \frac{g_{\text{view}}(k) - \mu_{\mathcal{C}}(g_{\text{view}})}{\sigma_{\mathcal{C}}(g_{\text{view}}) + \varepsilon}. \quad (36)$$

The normalized gains are converted into soft targets using a sigmoid with temperature T :

$$\tau_{\text{view}}(k) = \sigma\left(\frac{\tilde{g}_{\text{view}}(k)}{T}\right). \quad (37)$$

The view policy outputs one logit $\hat{\ell}_{\text{view}}(k)$ for each candidate viewpoint $k \in \mathcal{C}$. It is trained with binary cross entropy over the candidate-view logits:

$$\mathcal{L}_{\pi_{\text{view}}} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} \text{BCEWithLogits}\left(\hat{\ell}_{\text{view}}(k), \tau_{\text{view}}(k)\right). \quad (38)$$

Push policy. The push policy is trained analogously using candidate-level push gains $g_{\text{push}}(p)$. The gains are normalized over valid push candidates and converted into sigmoid soft targets:

$$\tilde{g}_{\text{push}}(p) = \frac{g_{\text{push}}(p) - \mu_{\mathcal{P}_t}(g_{\text{push}})}{\sigma_{\mathcal{P}_t}(g_{\text{push}}) + \varepsilon}, \quad \tau_{\text{push}}(p) = \sigma\left(\frac{\tilde{g}_{\text{push}}(p)}{T}\right). \quad (39)$$

The push policy outputs one logit $\hat{\ell}_{\text{push}}(p)$ for each candidate push $p \in \mathcal{P}_t$. With valid-candidate mask $M(p)$, the masked binary cross entropy loss is

$$\mathcal{L}_{\pi_{\text{push}}} = \frac{1}{\sum_p M(p) + \varepsilon} \sum_{p \in \mathcal{P}_t} M(p) \text{BCEWithLogits}\left(\hat{\ell}_{\text{push}}(p), \tau_{\text{push}}(p)\right). \quad (40)$$

This objective trains the policy to learn relative candidate quality within the current push set, rather than a calibrated absolute gain across different scenes.

Belief map estimator. The belief map estimator is trained with an occupancy loss on the Beta mean and a semantic cross-entropy loss. Let

$$\mu_t^{\text{occ}}(v) = \frac{\alpha_t(v)}{\alpha_t(v) + \beta_t(v)} \quad (41)$$

denote the predicted occupancy probability. We optimize

$$\mathcal{L}_{f_{\text{scene}}} = \frac{\sum_v M_t(v) (\mu_t^{\text{occ}}(v) - y^{\text{occ}}(v))^2}{\sum_v M_t(v) + \varepsilon} + w_{\text{sem}} \frac{\sum_v M_t(v) \text{CE}(\hat{s}_t(v), y^{\text{sem}}(v))}{\sum_v M_t(v) + \varepsilon}, \quad (42)$$

where M_t is the valid supervision mask. For the scratch checkpoint used by the main model, we set $w_{\text{sem}} = 0.1$. For class-ODD finetuning, we start from this pretrained $n_{\text{sem}} = 15$ checkpoint and use the finetuning default $w_{\text{sem}} = 1.0$. The semantic prediction is fed back at the next step as a detached softmax distribution.

B.4 Training Procedure

All modules are trained in simulation and selected by validation performance. DiVE and the two mode-specific policies are trained on low- and high-occlusion scenes. Extreme-occlusion scenes are not used for training and are reserved for held-out evaluation. For the class-OOD experiment, only the belief map estimator is lightly finetuned for the newly introduced semantic classes; DiVE and the action-selection policies remain frozen.

For DiVE, we use teacher forcing at the beginning of training and linearly anneal the teacher-forcing probability to zero, so the model gradually transitions to self-feeding its own visibility predictions. The push-resolvable branch self-feeds throughout training because its target is defined as a marginal action-induced visibility-change map rather than a temporally accumulated view-frequency state.

For policy learning, candidate gains are not regressed directly. Instead, each candidate set is normalized independently using z -score normalization and converted into sigmoid soft targets with temperature $T = 1.0$. This trains the policies to predict relative candidate quality within the current decision state, which is the quantity needed at inference.

For the belief map estimator, we first train a scratch checkpoint on low- and high-occlusion YCB scenes with $n_{\text{sem}} = 15$ and $w_{\text{sem}} = 0.1$. For the class-OOD setting, we finetune only f_{scene} from this checkpoint on unseen-v1 data using the reserved semantic slots, while keeping DiVE and both action-selection policies frozen. In both stages, the previous semantic belief is fed back as a detached softmax distribution, which stabilizes recurrent belief updates over long action sequences. The best checkpoint is selected using validation reconstruction quality, with occupancy IoU used as the primary selection criterion.

C Experimental Details

C.1 Baselines

We compare DiVE with three reconstruction baselines and one visibility-decomposition ablation. All methods are evaluated under the same voxel grid, candidate viewpoint set $\mathcal{C}$, candidate push set $\mathcal{P}_t$, object categories, action budget, and evaluation protocol. Unless otherwise stated, all learned components are trained on the same simulation dataset described in Sec. B.1. This ensures that the comparison isolates the effect of action selection and visibility decomposition rather than differences in scene distribution, sensing geometry, or evaluation budget.

CNABU. CNABU is a belief-gain planning baseline based on the map-space belief prediction framework of Marques et al. [9]. It represents the scene belief as a metric-semantic map and uses learned belief updates to predict the effect of sensing and manipulation actions. At each decision step, CNABU evaluates candidate view and push actions by estimating their predicted post-action beliefs and scoring them with volumetric information gain. The action with the highest predicted gain is then selected.

In our experiments, CNABU is trained on our dataset and evaluated with the same action budget, candidate viewpoint set $\mathcal{C}$, and candidate push set $\mathcal{P}_t$ as DiVE. Unlike DiVE, CNABU does not use decomposed view-resolvable and push-resolvable visibility maps. Instead, it relies on candidate-wise post-action belief prediction and information-gain evaluation. This makes CNABU a strong belief-gain planning baseline, but also computationally more expensive at test time because action selection requires evaluating the predicted gain of candidate actions online.

View-Only. View-Only is an ablation of DiVE that removes physical interaction from the action space. At every step, the planner is forced to select the viewing mode and chooses the next camera pose using π_{view} over the candidate viewpoint set $\mathcal{C}$. The push policy π_{push} is never invoked, and no swept volume is passed to the belief update. All other components, including the view-resolvable visibility update and the scene belief estimator, are kept unchanged.

This baseline isolates the role of pushing. If hidden regions can be recovered by sensing alone, View-Only should approach DiVE. However, in high- and extreme-occlusion scenes, some regions are not visible from any accessible viewpoint until the scene geometry is physically changed. In those cases, View-Only is expected to plateau because it can continue moving the camera but cannot remove occluders or expose physically hidden objects.

Random. Random is included as a lower-bound exploration baseline. It uniformly samples the next viewpoint from $\mathcal{C}$ without using DiVE, CNABU, belief gain, visibility maps, or any learned action-selection policy. The resulting observation is fused into the scene representation using a classical occupancy update. Because Random does not reason about which regions are unresolved or which actions are likely to reveal hidden space, it measures the benefit of informed active exploration over uninformed viewpoint sampling.

We use Random only as a weak exploration reference, not as a direct architectural ablation of DiVE. In particular, poor Random performance should not be interpreted as evidence for any single component of DiVE; rather, it indicates the gap between active visibility-aware planning and uninformed sensing.

Unified visibility ablation. Unified visibility is a visibility-decomposition ablation that removes the separation between view-resolvable and push-resolvable signals. Instead of predicting two modality-specific maps, the model predicts a single visibility map ϕ_{unified} that combines the remaining view-resolvable visibility change and the push-resolvable visibility change.

For this ablation, we construct the unified supervision target as

$$\phi_{\text{unified},t}^*(v) = \phi_{\text{view-rem},t}^*(v) + \phi_{\text{push},t}^*(v), \quad (43)$$

where the remaining view-resolvable component is defined as

$$\phi_{\text{view-rem},t}^*(v) = \left| \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} m_k(v) - \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} m_k(v) \right|. \quad (44)$$

Here, the first term is the marginal visibility of voxel v over all candidate viewpoints, and the second term is the visibility already accumulated from the visited viewpoints. Their absolute difference measures the remaining discrepancy between the current view-resolvable visibility state and the full candidate-view marginal visibility. The push component $\phi_{\text{push},t}^*$ is the same target used to train the push-resolvable head in DiVE.

The same training data, voxel grid, candidate viewpoint set, and candidate push set are used as in DiVE, but the two visibility heads are replaced by a single output head. This ablation tests whether the performance gain comes merely from predicting a visibility-like map, or from preserving the action modality responsible for making each region visible. A unified map can indicate that a region is visibility-relevant, but it does not specify whether that region is better resolved by camera motion or by physical interaction.

C.2 Action-Space and Planner Sensitivity Details

We vary one action-space parameter at a time while keeping all other components fixed. The left table in Fig. ?? reports the change in final occupancy IoU, in percentage points, relative to the default configuration of 98 viewpoints, 20 push candidates, a 15 cm push length, and a $\pm 30^\circ$ direction range. Performance changes remain small across the tested alternatives, indicating that the gains are not tied to a single candidate-set parameterization.

We also compare the gain-based mode selector with fixed schedules that push every 3, 5, or 7 actions (Fig. ??, right). All fixed schedules reduce final IoU on high-occlusion scenes. Increasing the post-push warm-up from the default of two accumulated views to three, four, or five views also provides no improvement, supporting the two-view constraint used in the main experiments.

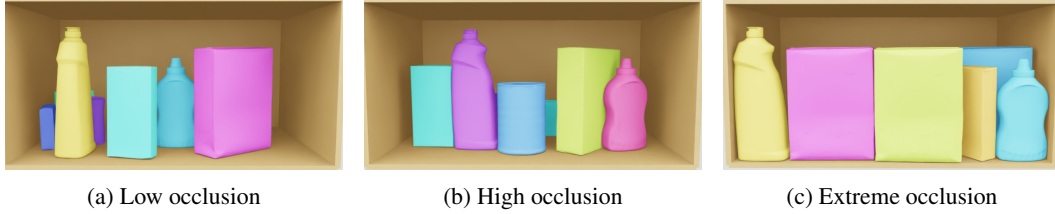


Figure 6: Example simulation scenes from the three in-distribution occlusion regimes. Low-occlusion scenes contain sparse object arrangements where most objects are visible from at least one viewpoint. High-occlusion scenes use denser placements with stronger mutual occlusion. Extreme-occlusion scenes further increase packing density, causing several objects to be fully hidden from all candidate viewpoints before interaction.

C.3 Test Scene Generation

We evaluate simulation performance on held-out Isaac Sim test scenes constructed from YCB objects. Each scene is captured from the same set of 98 candidate viewpoints used by the planner, with RGB-D observations and ground-truth voxel-level occupancy and semantic annotations. All test scenes are generated from random seeds that are disjoint from the training and validation sets.

The in-distribution test scenes are designed to vary occlusion difficulty while keeping the object category set and object geometry fixed. We use three occlusion regimes: low, high, and extreme occlusion. Figure 6 shows representative simulation scenes from these three occlusion regimes.

Low-occlusion scenes. Low-occlusion scenes are generated using the same YCB category set and a placement regime close to the training distribution. Each scene contains 11 active objects placed with collision-free sampling and sufficient inter-object spacing. As a result, most object instances are visible from at least one of the candidate viewpoints. On average, only 1.08 objects per scene are fully hidden from all 98 views, with at most 3 fully hidden objects in a scene. This split measures performance under the standard visibility regime in which most hidden regions can be recovered by viewpoint changes.

High-occlusion scenes. High-occlusion scenes use the same object pool and capture protocol, but the placement sampler is tightened to create denser configurations with more frequent contact, stacking, and mutual occlusion. Each scene again contains 11 active objects, but the average number of objects fully hidden from all candidate views increases to 1.44, with a maximum of 5. This split tests whether the model remains effective when larger portions of the scene are not visible from individual viewpoints and the belief estimator must reason over more occluded volume.

Extreme-occlusion scenes. Extreme-occlusion scenes form the main stress-test regime. They are generated with even denser object placement and contain 12 active objects per scene, one more than the low- and high-occlusion splits. These scenes frequently contain strong image-plane overlap and objects placed behind other objects. On average, 4.28 objects per scene are fully hidden from all 98 candidate views, with 4 to 5 fully hidden objects in every scene. This split evaluates performance under severe clutter, where successful reconstruction requires stronger amodal completion and occlusion-aware belief estimation.

Together, these three in-distribution splits form an occlusion-difficulty ladder: low-occlusion scenes evaluate standard active reconstruction, high-occlusion scenes increase the amount of hidden geometry, and extreme-occlusion scenes test whether the model can maintain useful beliefs when several objects are completely unobservable before interaction.

C.4 Out-of-Distribution Evaluation

We further evaluate generalization using two out-of-distribution test splits. Both splits keep the placement regime close to the low-occlusion setting so that the geometric difficulty remains comparable to the in-distribution low-occlusion split. This allows us to attribute performance changes mainly to object-level distribution shift rather than to increased occlusion.

Class-OOD scenes. The class-OOD split uses the same low-occlusion placement regime, but replaces the training object categories with held-out YCB categories that do not appear in the training set. Specifically, we use two unseen categories, 024_bowl and 025_mug, which are excluded from the in-distribution YCB object set in Table 3. These categories introduce semantic and geometric structures not observed during training, including a concave bowl shape and a mug handle. Each scene contains 9 active objects, reflecting the smaller held-out category pool. The average number of objects fully hidden from all candidate views is 1.08. Since the occlusion level is close to the in-distribution low-occlusion split, this evaluation isolates generalization to unseen object categories rather than increased occlusion difficulty.

Shape-OOD scenes. The shape-OOD split uses the same in-distribution category set and the same low-occlusion placement regime, but changes the object geometry by applying anisotropic per-axis scaling to each object instance. Specifically, the scale factor for each object axis is sampled uniformly from $[0.8, 1.2]$, producing geometric variations that are not seen during training while preserving the original category identity. Each scene contains 11 active objects. The average number of fully hidden objects is 1.12, keeping the occlusion difficulty close to the in-distribution low-occlusion setting. This split evaluates whether the model depends on memorized object shapes or can generalize its visibility and belief predictions to geometric variations of known categories.

The class-OOD and shape-OOD splits are complementary. The former changes semantic identity while keeping geometry relatively standard, whereas the latter keeps category identity fixed while changing geometry. Comparing them helps separate degradation caused by category-level priors from degradation caused by shape-level priors.

C.5 Real World

C.5.1 Real-World Setup

We deploy DiVE zero-shot on a real robot shelf setup. The physical setup uses a Piper AgileX manipulator positioned in front of a shelf with geometry comparable to the simulation environment. An RGB-D camera is mounted on the robot end-effector and provides partial observations of the shelf scene from selected viewpoints. The robot executes the same type of view-and-push exploration policy as in simulation: at each step, it either moves the camera to a selected viewpoint or executes a pushing action to alter the occlusion structure. Figure 7 shows the real-world robot setup and the physical object set used in the experiments.

Robot and camera. The robot platform is a Piper AgileX manipulator. An Intel RealSense LiDAR Camera L515 is mounted near the end-effector and provides RGB-D observations from the selected viewpoints. Camera poses are calibrated with respect to the robot base frame, allowing observations from different viewpoints to be integrated into the global voxel grid. The shelf geometry is chosen to be comparable to the simulation workspace, so the same voxelized reconstruction pipeline and view-and-push action structure can be used without retraining the planner.

Object set. The real-world scenes are constructed using physical objects that correspond to the YCB categories used in simulation. Although the category labels are shared with simulation, the real objects differ in exact size, shape, texture, reflectance, and material. This makes the real-world experiment a zero-shot transfer test for the visibility-based planner under sensing noise and appearance shift.



(a) Real-world robot and shelf setup.



(b) Physical objects used in real-world scenes.

Figure 7: Real-world experimental setup. The robot observes a cluttered shelf using an end-effector-mounted RGB-D camera and performs viewpoint changes or pushing actions selected by the planner. The object set consists of physical instances corresponding to the YCB-category objects used in simulation, but with real-world differences in shape, material, texture, and sensing noise.

Scene construction. We construct 10 real-world cluttered shelf scenes. Each scene contains multiple YCB-category objects arranged to create partial occlusions. Some objects are visible in the initial observation, while others are partially or fully occluded by front objects. The initial arrangement is manually created but not optimized for any method.

Observation and reconstruction. At each step, the RGB-D observation is converted into the same voxel grid representation used in simulation. The belief map estimator updates the global scene belief from the current observation. After a pushing action, the swept volume of the push is also passed to the belief update so that the estimator can account for the region physically affected by the interaction.

Execution protocol. Each real-world episode uses a fixed budget of 20 actions. We compare DiVE with View-Only under the same initial scene configurations. DiVE may choose either viewpoint changes or pushes, whereas View-Only is restricted to camera motion. No real-world finetuning is performed; the deployed policy is transferred zero-shot from simulation.

Real-world evaluation. For real-world episodes, we evaluate reconstruction at the object level rather than with dense voxel-level ground truth, since precise voxel annotation is unreliable under real sensing noise, calibration error, and manipulation-induced object motion. After each episode, we manually inspect the scene and record whether each object is correctly discovered, geometrically found but semantically misclassified, or missed. For initially occluded objects, we additionally record whether they become discovered during exploration.

D Result Details

D.1 Qualitative Rollout Analysis

Figure 8 visualizes a representative internal rollout of DiVE in simulation. For view steps, we show the RGB observation, the view-resolvable visibility map ϕ_{view} , the push-resolvable visibility map ϕ_{push} , and the resulting occupancy belief map. For push steps, we visualize the swept volume of the selected push action, which is provided to the belief update after interaction.

The rollout illustrates how DiVE alternates between sensing and interaction during exploration. In the early steps, the planner selects view actions while the scene still contains regions that can be resolved from additional viewpoints. After several observations, the planner selects a push action, and the corresponding swept map indicates the region physically affected by the interaction. The belief map is then updated after the push, allowing subsequent view actions to refine the reconstructed scene. A second push is selected later in the episode, showing that DiVE can repeatedly return to interaction when the remaining scene structure requires physical rearrangement.

This qualitative example complements the main quantitative results by showing how decomposed visibility maps and push-conditioned belief updates evolve over an exploration episode. While this visualization does not show the candidate-level policy scores, it demonstrates that the selected action sequence produces meaningful changes in the belief map, especially after push actions.

D.2 Hierarchical Action Selection Analysis

To further analyze action selection, we visualize the push-candidate set and the resulting policy decision at the same simulation step. Figure 9 first shows the 20 push candidates considered at this decision step. Each candidate is represented by its swept map, which marks the voxels that would be physically affected if the corresponding push were executed. This visualization clarifies the input space of the push policy: π_{push} does not directly optimize over a continuous interaction space, but ranks a finite set of sampled push candidates whose swept regions overlap different parts of the current scene.

Figure 11 then compares DiVE and the View-Only baseline at this same decision step. While Fig. 8 visualizes how the internal maps evolve over time, this comparison focuses on how the planner converts the visibility signals into a specific action choice. In the DiVE panel, the push-candidate heatmap summarizes the policy scores assigned to the candidates shown in Fig. 9, and the selected candidate is visualized through its swept region.

At this scene state, the mode-level expected-gain comparison indicates that pushing is more informative than viewing, i.e., $G_{\text{push}}(\phi_{\text{push},t}) > G_{\text{view}}(\phi_{\text{view},t})$. DiVE therefore selects the pushing mode, ranks the candidate pushes using π_{push} , and executes the selected action whose swept region is shown in Fig. 11. In contrast, the View-Only baseline is restricted to viewpoint changes and is therefore forced to select another view, even though the push-resolvable gain is larger at the same state.

The resulting belief maps highlight the practical consequence of this difference. DiVE’s push action updates the unresolved region more effectively by physically altering the occlusion structure, whereas the additional viewpoint selected by View-Only provides more limited improvement. This qualitative comparison complements the quantitative results in the main paper by showing that DiVE not only decides when interaction is needed, but also benefits from selecting the appropriate action mode before ranking candidates within that mode.

D.3 Unified Visibility Ablation Analysis

We further analyze the unified visibility ablation by examining whether the predicted map provides a reliable candidate-ranking signal for push actions. For each push candidate p with swept volume Ω_p , we compute a swept-volume score by summing the predicted visibility response over the voxels affected by the push:

$$s_{\text{unified}}(p) = \sum_{v \in \Omega_p} \phi_{\text{unified}}(v), \quad s_{\text{push}}(p) = \sum_{v \in \Omega_p} \phi_{\text{push}}(v). \quad (45)$$

Here, $s_{\text{unified}}(p)$ is computed from the unified visibility map, whereas $s_{\text{push}}(p)$ is computed from the decomposed push-resolvable map. We compare both scores against the true push-induced visibility gain over the candidate push set.

Figure 12 shows the distribution of per-sample Pearson correlations between the swept-volume scores and the true push gain. The decomposed push-resolvable map produces a distribution that

is shifted toward positive correlation, indicating that high responses in ϕ_{push} tend to overlap with push candidates that actually produce large visibility changes. In contrast, the unified map produces a broader distribution with substantially more weak or negative correlations. This means that the unified map often assigns high response to regions that are visibility-relevant in a general sense, but not necessarily useful for selecting a push action.

This result highlights the role of visibility decomposition in candidate-level action selection. A single unified map can represent that a region remains visibility-relevant, but it does not preserve which action modality is responsible for resolving that region. Consequently, swept-volume scores computed from the unified map can be contaminated by view-resolvable regions that should be handled by camera motion rather than physical interaction. The decomposed push-resolvable map avoids this ambiguity by retaining a push-specific signal, making candidate push ranking more aligned with the actual effect of interaction.

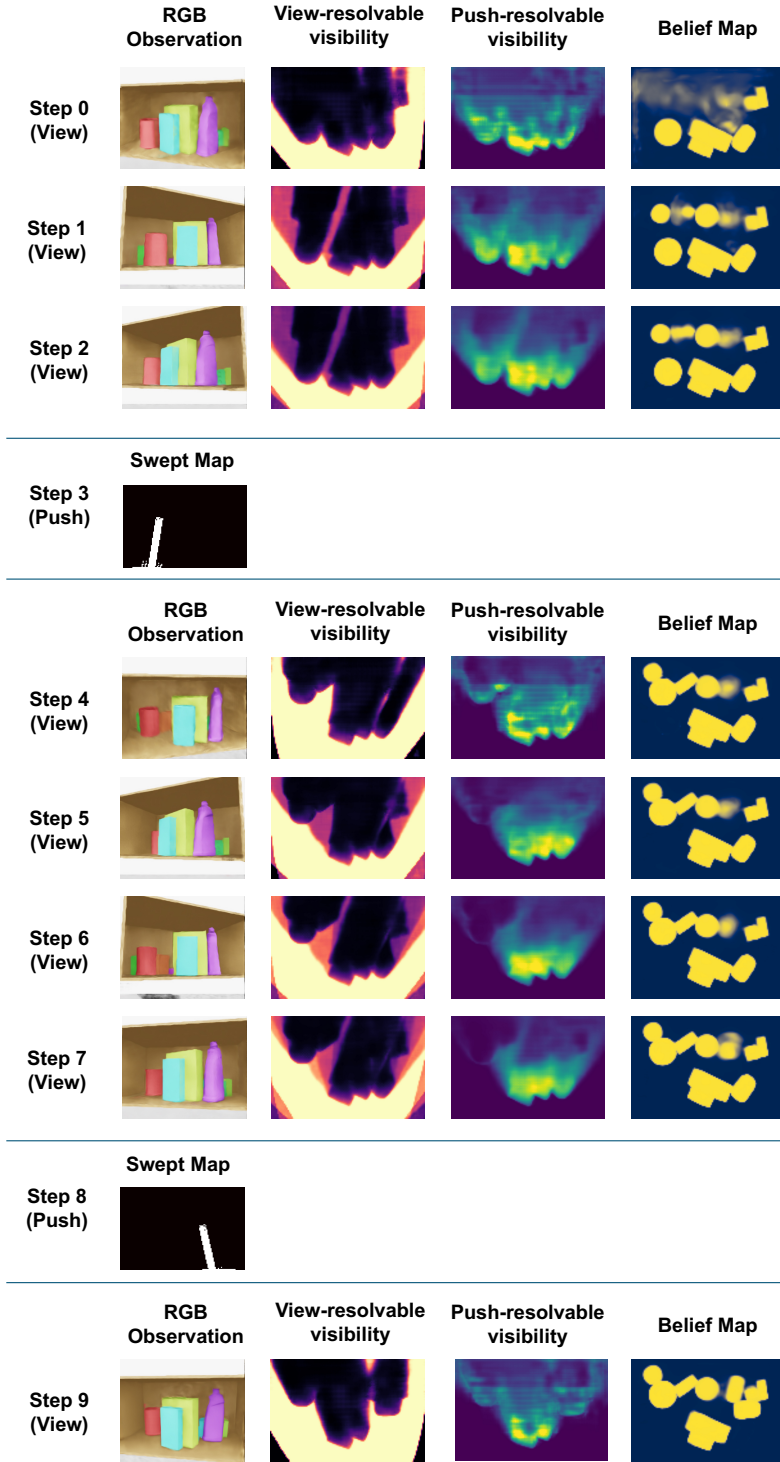


Figure 8: Qualitative internal rollout of DiVE in simulation. For view steps, each row shows the RGB observation, the top-down projection of the view-resolvable visibility map ϕ_{view} , the push-resolvable visibility map ϕ_{push} , and the occupancy belief map. For push steps, we show the swept map of the selected push action, which marks the region physically affected by interaction. The sequence illustrates how DiVE alternates between viewpoint changes and pushing: view actions refine visible and view-resolvable regions, while push actions modify the scene and update the belief through the swept region.

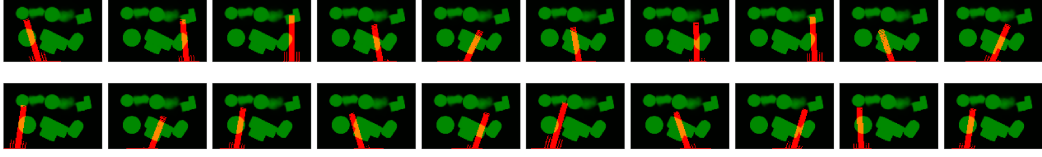


Figure 9: Visualization of the 20 push candidates considered at the decision step analyzed in Fig. 11. Each panel shows the swept map Ω_p of one candidate push over the current scene, indicating the region that would be physically affected by executing that push. The push policy π_{push} ranks these candidates within the selected pushing mode and chooses the candidate predicted to produce the largest push-induced visibility gain.



Figure 10: Visualization of the 98 candidate viewpoints considered at the decision step analyzed in Fig. 11. Each panel shows the RGB observation that would be obtained from one candidate viewpoint of the current scene. The view policy π_{view} ranks these candidates within the viewing mode and selects the viewpoint predicted to yield the largest view-resolvable visibility gain.

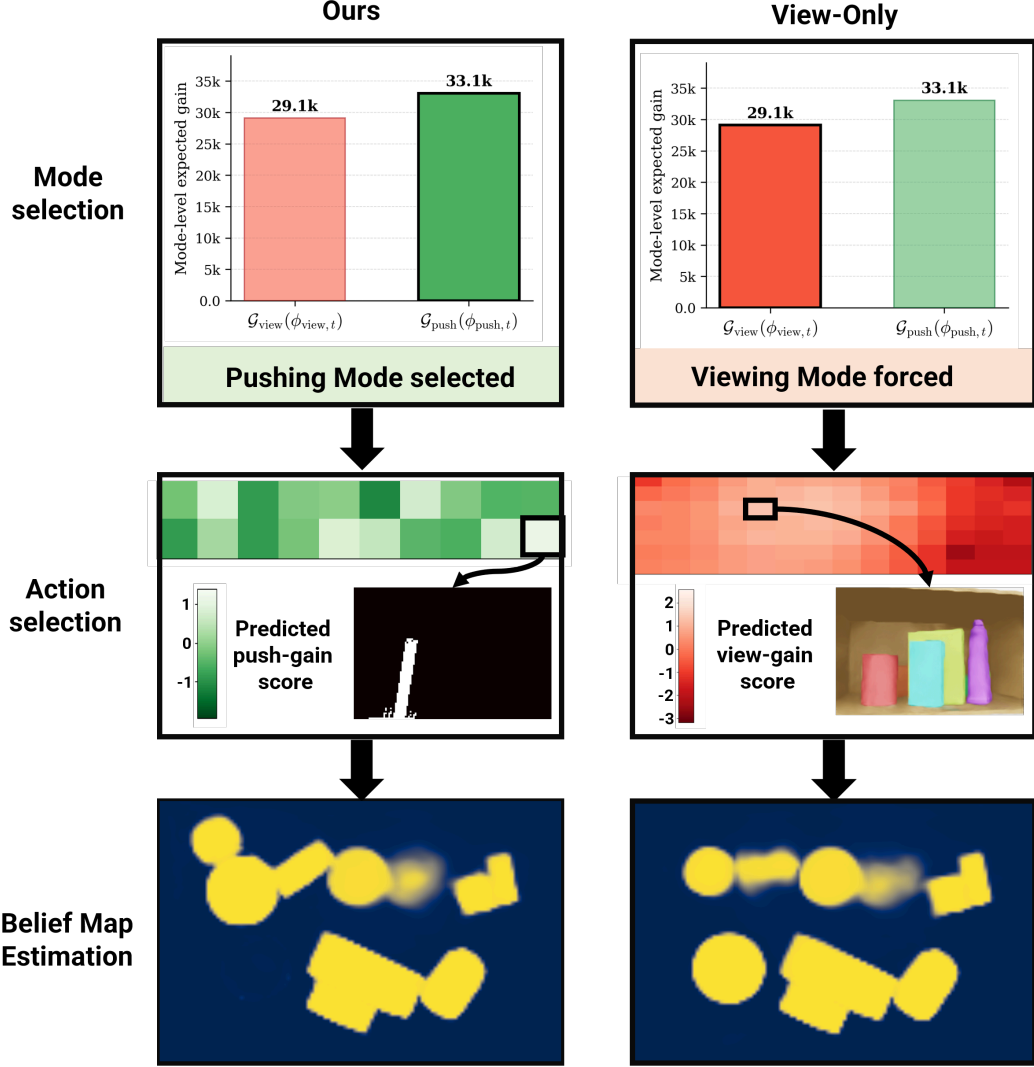


Figure 11: Candidate-level action-selection comparison at the same decision step in simulation. Left: DiVE (ours). The mode-level expected-gain comparison favors pushing, so the planner selects the pushing mode, ranks the push candidates shown in Fig. 9, and executes the selected push, whose swept region is shown. Right: View-Only. Under the same scene state, the baseline is restricted to viewpoint changes and is therefore forced to select an additional view from the candidate viewpoints visualized in Fig. 10, despite the larger push-resolvable gain. The updated belief maps show that DiVE’s push action reveals and updates previously unresolved regions more effectively than an additional viewpoint alone.

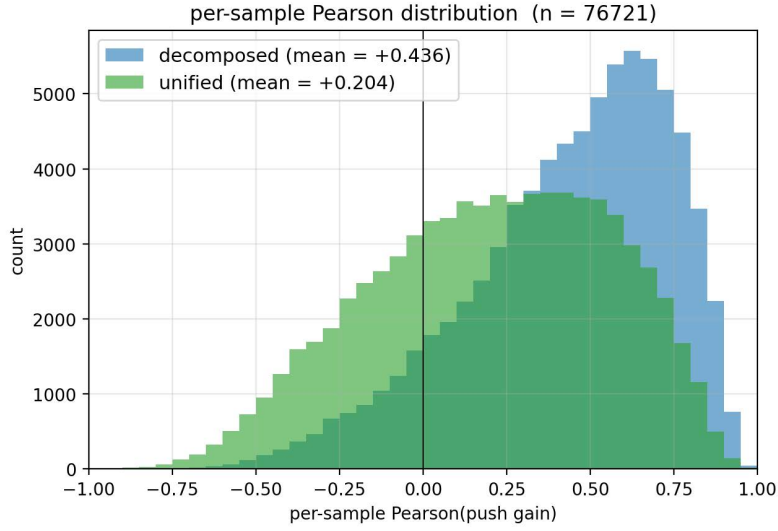


Figure 12: Distribution of per-sample Pearson correlations between swept-volume map scores and true push-induced visibility gain. The decomposed push-resolvable map ϕ_{push} yields a distribution shifted toward stronger positive correlations, showing that it provides a more reliable push-candidate ranking signal. The unified visibility map produces a broader distribution with more weak or negative correlations, indicating that mixing view-resolvable and push-resolvable signals dilutes the push-specific cue required for interaction selection.